Visualization Autocomplete: Visualization Authoring via Stepwise Design Recommendations

Hyeon Jeon, Sungbok Shin, and Niklas Elmqvist

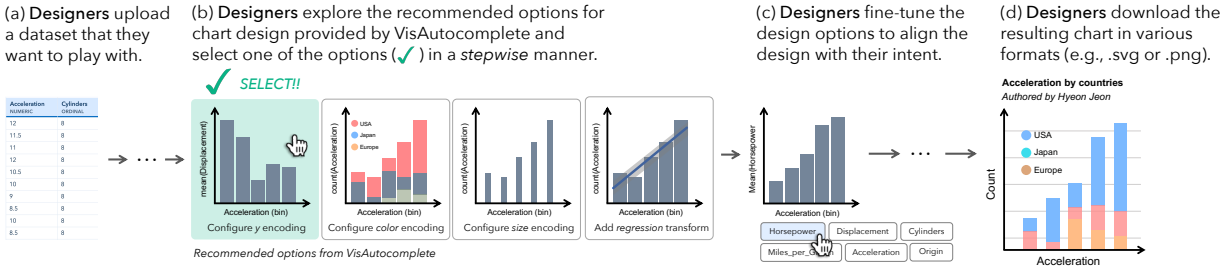

Fig. 1: **Stepwise visualization design with VisAutocomplete.** VisAutocomplete is a novel visualization authoring system that supports stepwise design autocompletion using recommendations. The workflow starts with authors uploading a dataset (a). For each step, the system recommends an ordered list of recommendations to guide the visualization design, where designers can select one among them (b). Some recommendations have user-controllable parameters, such as the number of bins, the color scale, or the chart title (c) for minor adjustments. Once the visualization is complete, it can be exported as either a specification or an image file (d).

Abstract—When domain experts create charts, the bottleneck is rarely the data, but knowing the optimal next step in chart design. The visualization design space is vast, and while domain experts can recognize a good design when they see it, it is often challenging to determine the exact path to get there. To address this, we present VISAUTOCOMPLETE, a system inspired by text autocompletion that reconceptualizes visualization design as a sequential process, recommending concrete next steps at each stage of the authoring process based on common practices. Users can intervene at any step, or delegate multiple steps to the system and select one from the design recommendations. To support responsive interaction, we distill the translation logic of a large language model (LLM) into a single function that receives the current chart state and recommended transition as input and returns the updated chart specification as output. We evaluate the system against a LLM vibecoding, Microsoft Excel, and TaskVis, an automated chart recommendation engine, on chart quality and approachability. Our results show that VisAutocomplete outperforms all baselines in the articulacy of complex chart authoring, while remaining on par with LLM in approachability.

Index Terms—Visualization authoring, visualization design process, visualization recommendation, human-centered AI, human agency, automation, mixed-initiative interaction.

1 INTRODUCTION

Effective data visualization is essential for communicating insights quickly and accurately in many disciplines, ranging from science to finance, medicine to mechanical engineering, office work to journalistic inquiry. However, domain expertise does not necessarily translate to fluency in creating visualizations [11]. All these decisions, such as choosing the best chart type, variable encoding, color scale, axis layout, or data legend, can easily become overwhelming in the face of a massive array of design possibilities. While automated visualization design tools exist [26, 42, 54], and a subset of these tools have sought a balance between agency and automation [60], they tend to require specialized knowledge and notation, and yield resulting visualizations with little opportunity for design tweaking [20, 49, 52] or learning [25] from the process for next time. Interactive chart design tools [36, 62], on the other hand, place a considerable burden on users and consequently do not tend to integrate best practice recommendations in their workflows.

To address this gap, we propose *stepwise visualization autocompletion*, an approach that reconceptualizes visualization authoring as a step-by-step process (Figure 1, 2). Inspired by text autocompletion that interactively suggests words and phrases as users type, our approach guides users through consecutive visualization design decisions. We model each decision as a valid visualization specification. At each step, the system recommends a set of possible next states from the current state. These are derived from design patterns observed in large-scale visualization corpora.

Visualization autocompletion achieves two key qualities simultaneously: **approachability** and **articulacy**. Automated tools prioritize approachability for domain experts by generating complete visualizations in a single step. However, this coarse granularity constrains articulacy, offering little support for delicate expression. In reality, experts may not always have a clear vision of the final output, and visualization design is inherently an iterative process of exploring possibilities and progressively refining choices. Visualization autocompletion achieves both simultaneously by keeping the process approachable by presenting an ordered list of candidate states, and pursuing articulacy by integrating data-driven recommendations with user choice.

We validate visualization autocompletion through a web-based system called VISAUTOCOMPLETE[1]. VisAutocomplete guides users through chart creation as a structured, step-by-step authoring process, progressively selecting each component of the visualization specification through recommendations, including data encodings, transformations, and visual styling. At each stage, candidate changes are visually previewed and ranked by common practices, which users may

- *Hyeon Jeon is with Seoul National University, Seoul, South Korea. E-mail: hj@hcil.snu.ac.kr.*
- *Sungbok Shin, the corresponding author, is with Sogang University, Seoul, South Korea. E-mail: sbshin90@sogang.ac.kr.*
- *Niklas Elmqvist is with Aarhus University, Aarhus, Denmark. E-mail: elm@cs.au.dk.*

Manuscript received xx xxx. 201x; accepted xx xxx. 201x. Date of Publication xx xxx. 201x; date of current version xx xxx. 201x. For information on obtaining reprints of this article, please send e-mail to: reprints@ieee.org. Digital Object Identifier: xx.xxxx/TVCG.201x.xxxxxxx

[1] https://vis.autocomplete.studio/

accept outright, refine further, or delegate across multiple steps at once. Recommendations are generated in real-time without requiring on-the-fly LLM inference. We implement this by constructing a translation function that takes the current chart specification and a recommended transition as input and returns the updated chart specification. This function is authored by the LLM, which we prompt to leverage its code-generation capabilities. At any point, users retain the ability to manually override individual parameters, ensuring that automation enables rather than constrains.

However, it remains an open question whether such a finer-grained, iterative approach can meaningfully improve articulacy while remaining approachable for domain experts without a background in data visualization. To evaluate VisAutocomplete across both articulacy and approachability, we compare visualizations produced with the system against those created with popularly used visualization authoring tools in practice: Microsoft Excel and vibecoding using ChatGPT. For an additional point of comparison, we include TaskVis [42], a chart recommendation engine inspired by Draco [26], as a representative of a fully automated baseline. We find that VisAutocomplete outperforms Microsoft Excel and automated solutions in terms of chart quality. While its performance is comparable to that of the LLMs on creating simple charts, VisAutocomplete demonstrates superior quality when constructing more complex charts. In terms of approachability, VisAutocomplete is either more approachable than, or at least on par with, the baselines.

2 Background

Below, we present the background for our research.

2.1 Visualization Design Process

Prior work has sought to characterize the visualization design process [4, 27, 40], and to understand how designers work in practice [11]. In particular, researchers have focused on domain experts in professional settings, who have little to no background in data visualization [58]. A significant proportion of visualization designers fall into this category, and this makes it essential to understand how they engage with the design process in order to better support them [9]. In practice, designers rarely begin with a clear vision of how their visualization should look [31], and the design process is further shaped by unexpected challenges and constraints [11, 40]. Consequently, arriving at an effective chart may require significant and iterative exploration of the design space until the designer is satisfied with the result [37].

Schön's notion of design as a *reflective practice* offers a key insight into how this iterative process actually unfolds: each decision is not planned in advance, but emerges from a *"situated conversation with the current design situation"* [31, 39]—that is, a designer's next move is always a local, grounded response to what they see in front of them at that moment. This suggests that while the overall design process may appear nonlinear and unpredictable, it may fundamentally unfold as a sequence of locally situated decisions, each informed by the current state of the design, much like a greedy traversal of the design space, one step at a time. This, in turn, suggests the need for a new model that supports the navigation of this iterative process toward a visualization that meets the designer's communicative goals.

Contribution. Inspired by Schön's notion of reflective practice [39], we model visualization design as a stepwise process.

2.2 Augmenting Visualization Design

Several directions have been suggested to help analysts in effective visualization design. One line of research automates the design process by recommending charts based on the specifications of the dataset [17, 59]. While this approach reduces the designer's burden, it limits the exploration of the broader design space. A related line of work incorporates designer intent into the recommendation process. However, these tools are based on programming and specification, which still present a barrier for domain experts without programming knowledge [20, 26]. A third line of research enhances the expressivity of visualizations via interaction. These tools originally aim to help novice designers by not having to do programming, but instead through direct user manipulation [21, 33, 34, 36, 62]. However, they can overwhelm users with complex interfaces, as they are difficult to learn [37].

Motivated by this, L'Yi et al. [22] propose blended interfaces that link familiar and unfamiliar authoring interfaces to improve learnability. A different line of tools assists designers throughout the design process itself, such as by linting [14] or detecting mirages [24], and by automating design feedback [44–46] to detect issues in chart design. Another line of work supports authoring through iterative interaction with generated outputs, whether by comparing design variants [50] or offloading data transformation to an AI agent [53].

Contribution. Rather than recommending a final visualization or exposing users to a complex authoring interface, we reconceptualize visualization design as a sequential process consisting of primitive actions, and realize it for chart authoring. At each step, we recommend the most common next design choices, constructing the specification one decision at a time, keeping the interaction simple and learnable.

2.3 Agency and Automation

The role of agency in automated systems has been a longstanding topic. While early work argues that automation could handle repetitive tasks efficiently [23], researchers later raised concerns about overreliance on it. Sheridan delineates the boundaries of appropriate automation use [43], Horvitz argues for a synergistic integration of automation and direct manipulation [15], and Shneiderman argues that AI should augment human abilities while maintaining appropriate human oversight rather than pursuing full autonomy [48]. Amershi et al. propose design guidelines stressing user control and error recovery [2]. The visualization community has similarly sought a balance between agency and automation [12, 26, 60].

Contribution. VisAutocomplete preserves user agency throughout the stepwise design process: users may accept a recommendation, override individual parameters, or delegate multiple steps at once. This instantiates HCAI principles [48] of human oversight over automated processes.

3 Stepwise Visualization Autocompletion

Visualization design is an inherently iterative process [28, 31, 45]: designers begin with only a partial sense of their goal, progressively refine their choices, and eventually find an acceptable solution (Figure 2 left). Ideally, they also collect transferable knowledge about designing good charts in the process. Yet most existing tools force a trade-off: automated tools sacrifice agency in the name of speed, while manual tools demand expertise that domain experts lack [58]. This leads us to our research question: *How can we support the iterative nature of visualization design for novices, enabling meaningful agency while integrating best design practices and transferring visualization knowledge in the process?*

3.1 Design Goals and Requirements

We identify two complementary goals that any answer to this question must satisfy simultaneously. **Approachability** means that a novice with no background in data visualization can engage with the system immediately, without learning a specification language or mastering a grammar of graphics. **Articulacy** in our context, is how faithfully a chart designer realizes a given *communicative* intent (the target message) in a chart. The system enables nuanced, intentional expression, that users can steer toward the chart they have in mind rather than accepting whatever the system produces.

Our proposed method for *stepwise visualization autocompletion* achieves both simultaneously: keeping the process approachable by presenting an ordered list of candidate states, and pursuing articulacy by integrating data-driven recommendations with user choice. This yields five design requirements:

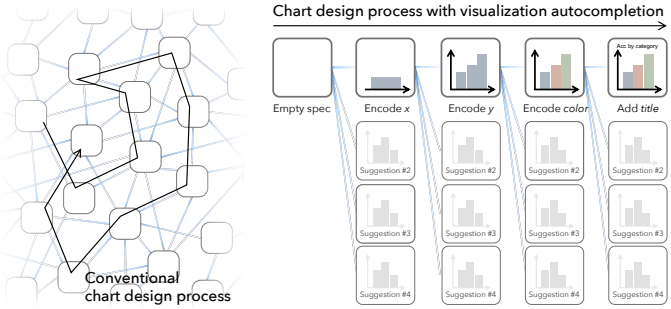

Fig. 2: **Recommendation as design space navigation.** Visualization autocompletion redefines visualization recommendation as ranked navigation in the entire space of valid visualization specifications.

DR1 **Responsive:** Recommendations must be generated in real time [47, 56], since the value of autocompletion collapses if the user must wait.

DR2 **Understandable:** The interface must require no prior knowledge of visualization grammars or specification languages.

DR3 **Anticipatory:** Users must be given a preview of how a recommendation will affect the visualization, so they can make an informed decision before committing to a choice.

DR4 **Trackable:** Users must be able to see where they are in the design process, and reflect how each step changed the chart.

DR5 **Controllable:** Users must be able to intervene at any step, modify the chart, delegate multiple steps at once, or override any parameter.

We note that *agency*, the ability to intervene, override, or direct the process, is not a separate goal but an emergent consequence of articulacy: a system that enables fine-grained expression necessarily gives users meaningful control.

3.2 Generalizing Autocompletion

Our approach is inspired by text autocompletion systems, which suggest probable and high-quality continuations as a user types a query or phrase [8]. At its core, autocompletion is a *ranked navigation method* through a combinatorial state space: the user anchors a starting point, and the system returns an ordered set of next states according to some model of quality or likelihood (Figure 2 right). This characterization is not specific to text: any domain that can be modeled as a space of valid states with ranked transitions is a candidate for autocompletion.

Text autocompletion, however, is a single-step interaction: the user selects one completion and issues the query. Applying the idea to visualization design means that the output is a *persistent artifact*, not a disposable search term. Because the designer refines an object rather than issuing a one-shot command, the interaction must support *multiple steps*, each one narrowing the design space toward a satisfying result. We argue that this is a natural generalization: autocompletion applies to any artifact-producing process where (1) the space of valid states is large and difficult to navigate unaided, (2) transitions between states can be ranked by quality, and (3) user intervention at any step is preserved.

3.3 Formalization: Navigating the Design Space

To formulate visualization design as a ranked navigation problem (Figure 2), we model the visualization design space as the set of all valid visualization specifications in a given grammar. Each valid specification V is a *state*, and a *transition* t moves from one valid state to another by making exactly one change. Formally, each transition t takes a visualization V and user-defined parameters θ and returns a new specification V', where $V' = t(V, \theta)$. For example, given a scatterplot V with x and y encodings already defined, a probable transition t is the addition of a color channel, where θ specifies the choice of encoded variable and color palette. The resulting structure is a directed graph over a large—potentially infinite—space; visualization autocompletion is a method for navigating it.

However, there may be a bewildering number of transitions t available for a given visualization state V. Visualization autocompletion is thus the problem of recommending, at each state, the highest-quality transitions available. To make the choice tractable to a novice user, we propose ranking the transitions based on some quality measure. Ideally, the top-ranked transition may help visualization designers make their charts better reflect the intended message. This means that the navigation of visualization design space using autocompletion can be viewed as an exploration of a Markov chain in which transition likelihood $P(V' \mid V, m)$ may reflect the quality of V' with respect to a target message m.

4 System: VisAutocomplete

We present VisAutocomplete, a visualization authoring system (Figure 3) that implements our proposed approach of stepwise visualization autocompletion (Section 3). We discuss our recommendation algorithm, an agent mode that automates multiple steps of recommendation, and the design of VisAutocomplete interface.

4.1 Recommendation Algorithm

VisAutocomplete relies on an algorithm that recommends a list of transition candidates for each visualization authoring step. We describe our recommendation criteria, then detail how we train and implement the criteria as an algorithm.

4.1.1 Recommendation Criteria

Given a visualization specification V, VisAutocomplete recommends the list of transitions t ranked by their quality. Though ideally the quality of t should be determined by the alignment between $V' = t(V, \theta)$ and the target message m, computationally assessing this alignment is nontrivial. We may leverage LLM's capability to understand chart semantics [19, 49], but this approach compromises execution time. Furthermore, designers often lack an explicit "message" to convey, making it inappropriate to require such input to the system.

We instead assess the quality of a transition t based on the *plausibility* of the transition, i.e., **the extent to which average visualization designers would commonly select it**. Our rationale is that, regardless of the target message m, such recommendations can guide authors toward visualizations that are likely to communicate effectively. Users can then evaluate whether these plausible recommendations align with their implicitly held message and adjust θ to improve this alignment.

4.1.2 Implementation

We detail how we implement our recommendation algorithm.

Visualization grammar. We use Vega-Lite [38] for two reasons. First, it provides a large, semantically rich design space for authoring visualizations [38, 59, 63], which enables VisAutocomplete to reflect the needs of diverse users and domains. Second, the specification structure consists of bricks of high-level *semantic primitives* [38, 51] (e.g., mark, encoding, style). This is not only easy to read [29] but also aligns well with our objective to predict 'plausible transition' of a given visualization spec [18, 63]. Still, we argue that the concept of visualization autocompletion is dataset-agnostic: it can be done with other codes or libraries, such as Matplotlib, or Plotly.js.

Base corpus. We utilize a Vega-Lite dataset collected by Ko et al. [19] to train our model. This dataset consists of 1,981 Vega-Lite specifications that represent real-world visualizations. We use this dataset because it is semantically diverse across multiple dimensions, including task complexity, chart type, and the presence of composite views. Such diversity makes it well-suited for uncovering how people commonly author visualizations (Section 4.1.1).

Recommendation engine. We build our recommendation engine by (1) decomposing visualization specifications into semantic primitives and (2) computing their co-occurrence patterns. Given a visualization V composed of primitives $T_V = \{t_1, t_2, \cdots, t_n\}$, the engine recommends new transitions $t \notin T_V$ that commonly co-occur with those in T_V. The detailed steps for training and computing the final recommendation score are as follows.

(Step 1) Decomposing visualization specifications. We analyze each Vega-Lite specification in our corpus, decomposing it into semantic primitives—atomic design choices within a visualization specification, such as a mark type (`{"mark": "bar"}`), a field encoding (`{"field": "year", "type": "temporal"}`), or a color scale (`{"scheme": "blues"}`). We iterate through two stages (details can be found in Appendix C):

Stage 1: Decompose the code of each chart specification into semantic primitives while ensuring that the resulting primitives can be categorized according to Wilkinson's Grammar of Graphics taxonomy [57]: *data, transform, scale, coordinate, element,* and *guide*. We design the deterministic rule for decomposing chart specifications also based on Vega-Lite documentation.

Stage 2: Merge primitives that share the same semantics across charts as a taxonomy (e.g., all instances that change the width of the charts (e.g., `continuousWidth` and `width`) are merged into a single primitive group). As with Stage 1, we design a rule for merging the primitives.

We iterate stages 1 and 2 by re-categorizing the semantic primitives in the chart corpus using the merged primitive set, and then merging semantically equivalent primitives again. We repeat the loop until the taxonomy of semantic primitives no longer undergoes changes through three iterations of the loop. Note that we do not use LLMs to directly generate primitives to reduce uncertainty and ensure consistency of the extraction. We rather use it to validate and refine the set of primitives extracted by our rules for chart decomposition and merging. When a problem is detected, we update the rules. This iterative loop can thus be interpreted as a variant of chain-of-thought prompting [55]. To further ensure consistency, we guide LLMs to refer to official Vega-Lite documentation using the model context protocol [16]. We use OpenAI GPT-5.2 to execute LLMs.

(Step 2) Computing co-occurrence. Given our corpus $\mathscr{V}$, where $|\mathscr{V}|$ denotes its cardinality (i.e., the total number of specifications in the corpus), we define the co-occurrence ratio $c(t_1,t_2)$ between primitives t_1 and t_2 as:

$$c(t_1,t_2) := \frac{|\{v \in \mathscr{V} \mid t_1 \in v \wedge t_2 \in v\}|}{|\mathscr{V}|} \tag{1}$$

We precompute the co-occurrence ratio of all pairs of primitives to reduce the runtime when users use the VisAutocomplete system.

(Step 3) Computing quality scores for recommendation. Given the current visualization V created by the designer through VisAutocomplete and its constituent primitive set V_T, The score of t is defined as its arithmetic mean of the co-occurrences with the primitives already present in V_T. Formally, it is represented as: $q(V,t) := \text{mean}\{\, c(t,t') \mid t' \in V_T \,\}$. Note that before any visualization is specified, the first recommendation is generated immediately after the user uploads a dataset. At this stage, V_T contains only the primitive representing the dataset type (e.g., tabular, time-series, or geospatial; classified as *data* in Wilkinson's taxonomy), and recommendations are made based on the initial choice made by users.

4.2 The Agent Mode

VisAutocomplete provides *the agent mode*, a multi-step autocompletion to reduce cognitive load and increase efficiency. We define *agents* as an automated process that proactively explores several steps ahead of users and recommends several resulting authoring paths. We develop agents with two objectives: (1) different agents should provide diverse recommendations, both in terms of the types of design choices made and the number of steps taken. (2) while doing so, agents should follow common visualization authoring practices in their recommendations. To this end, we control agent behavior via two hyperparameters ranging from 0 to 1: *randomness r* and *saturation threshold u*.

The *randomness* parameter controls the diversity in design choices. Formally, we define the probability of selecting recommendation $P_i(r)$ as a softened softmax:

$$P_i(r) := \frac{e^{s_i(1-r)}}{\sum_{j\in\{1,\ldots,n\}} e^{s_j(1-r)}},$$

where $s_1,\ldots,s_n$ denotes the quality scores of top-n recommendations. As r increases, the distribution becomes flatter, allowing the agent to explore lower-ranked recommendations more often. We randomly pick r ranging from 0 to 0.4 to prevent agents from outputting recommendations that do not follow the common practices.

The *saturation threshold u* determines when the iterative process should saturate and terminate. At each iteration, we quantify the difficulty of choosing among current recommendations using the entropy of the zero-randomness distribution $P_i(0)$:

$$H = -\sum_{i\in\{1,\ldots,n\}} P_i(0)\log P_i(0).$$

A higher entropy indicates that the recommendation scores are relatively uniform, suggesting that no transition is clearly preferred, whereas a lower entropy indicates that the choice is more concentrated on a few strong candidates. The iteration terminates when $H < u$. We randomly pick u within the range of 0.2 to 0.5. The range is determined heuristically.

When the agent mode is enabled, VisAutocomplete instantiates multiple agents and generates recommendations. Users can accept recommendations as is, apply them with minor adjustments such as modifying parameters (e.g., color palettes) or discarding specific steps, or dismiss them entirely.

4.3 Translating Recommendations into Vega-Lite Code

One technical challenge in VisAutocomplete is the translation of recommended transitions to Vega-Lite code. This is nontrivial because incorporating a new primitive into a Vega-Lite specification often requires more than simply appending a new leaf node; it also requires synchronizing with other parts of the specification. For example, when an x-axis is binned, adding a new variable to the y-axis requires determining how data points within each bin should be aggregated. Similarly, when multiple marks are combined through layering, the encoding of a new mark (e.g., combining `"confidence interval"` mark with `"line"` mark) must be determined in a manner that is consistent with the encodings of existing marks. One trivial way to implement this translation is to call an LLM, but this increases system latency (DR1).

To that end, we build a *LLM-distilled translation function* to address this problem. Our approach is to prompt the LLMs (OpenAI GPT-5.2) to autonomously design charts using VisAutocomplete, while the LLMs translate each recommended transition into an executable Vega-Lite code. For each translation, we store both the resulting code and the contextual information explaining why that translation was produced. We then distill these accumulated translations into a single JavaScript function that replicates the LLM's translation logic deterministically. The function receives a Vega-Lite spec V, a recommended transition t, and a parameter θ, then outputs a new Vega-Lite spec V' with the transition applied, i.e., $V' = t(V,\theta)$. By repeatedly running this process across diverse datasets, the function accumulates coverage over a wide range of design sequences, enabling VisAutocomplete to perform code translation responsively without on-the-fly LLM inference.

4.4 The VisAutocomplete Interface

We describe the VisAutocomplete interface (Figure 3).

Interactive spreadsheet. When users first upload their data, the system automatically shows it in an interactive spreadsheet and displays it in the upper-left panel of the interface. Users can then interactively explore the data and select the columns they wish to include in the visualization. After making their selections, they proceed to the authoring phase by clicking the 'Apply' button. The spreadsheet view is then automatically replaced with the chart view.

Recommendation panel. After users finish selecting the data columns of interest, the recommendation panel (Figure 3d) then presents top-K (in our case, K was 5, and can be extended) candidates for the next transition as interactive *recommendation cards* (Figure 3c). Initially, when the dataset is uploaded, the recommendation shows options to choose the marks. Afterward, it shows options related to data encoding (e.g., configure x encoding), data transformation (e.g., add *regression*

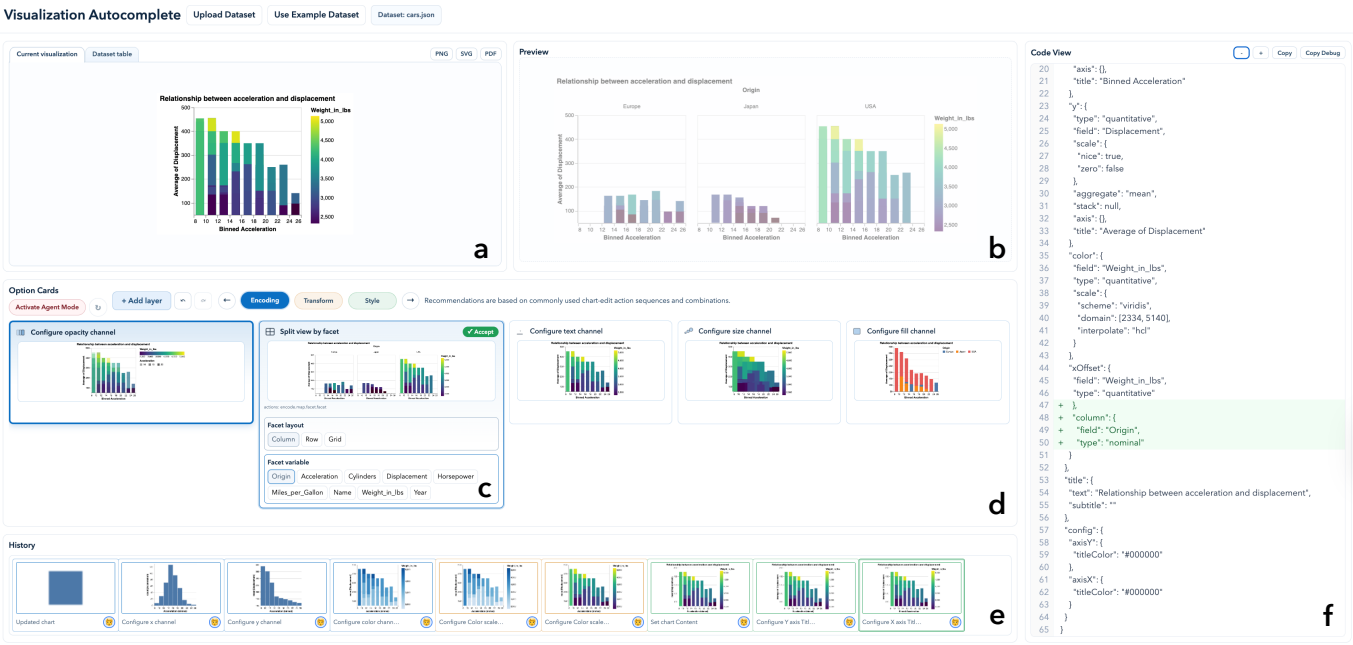

Fig. 3: **The VisAutocomplete interface.** Designers can update the *current visualization* (a) by exploring design recommendations presented in the *recommendation panel* (d). They can further tailor the visualization to their intent by adjusting the parameters of the *recommendation cards* (c). As they explore these options, the system *previews* the resulting visualization design (b) and shows how the visualization specification would change in the *code view* (f). Finally, designers can track how the visualization design evolves through the *graphical history view* (e).

transform), and style (e.g., configure chart *title*). Each card includes a thumbnail preview of the visualization produced by accepting the corresponding primitive, along with a short text label that summarizes the primitive's effect. Cards are ordered by descending score, with the highest quality score appearing at the top-left. To help users identify the most commonly chosen options at a glance, the top two cards are visually emphasized with bold borders and increased widths.

Accepting a recommendation is not an all-or-nothing action. Within each card, below the thumbnail, VisAutocomplete provides direct manipulation controls that users can use to adjust the parameters of the recommended primitive according to their favor (e.g., a recommendation to bin a quantitative field exposes a slider for the number of bins; a color-scale recommendation exposes a palette selector; a title recommendation exposes a text field). These controls let users accept a recommendation while still adjusting its details to their intent (DR5). In other words, the default values reflect best-practice guidelines, but can be overridden at any time. The controls appear only after a card is selected, keeping the panel uncluttered during browsing.

Note that the system first recommends chart marks, then allows users to request recommendations from one of three distinct categories: *encoding*, *data transformation*, and *style*. Encoding recommendations specify how attributes are mapped to visual channels, data transformation recommendations apply operations that modify the data directly, such as filtering or scaling, and style recommendations adjust aesthetic properties. We adopt this categorization to prevent users from being overwhelmed by recommendations with substantially different characteristics. This design also aligns with the stages described in the reference model of Card, Mackinlay, and Shneiderman [5].

Chart view and preview. The right side of the upper panel displays the current Vega-Lite visualization authored by users (Figure 3a), along with buttons to download it in various formats, including .png and .pdf. On the left side, the "preview" of the chart that will be confirmed when the recommendation card is clicked is displayed with lower opacity (Figure 3b). This view is updated instantly as the user hovers over recommendation cards or adjusts parameter controls (DR1). This anticipatory preview (DR3) allows users to evaluate candidate transitions without committing to them or having visualization knowledge (DR2). This capability reduces the cost of exploration.

Graphical history view. At the bottom of the interface, a horizontal, graphical history interface [13] (Figure 3e) provides a visual record of every design decision the user has made (DR4). Each entry is a thumbnail of the chart as it appeared after a given design step, ordered chronologically from left to right. The most recent state is anchored at the right edge of the visible area, so that the most recent states are visible without scrolling. Earlier states overflow to the left and are accessible by scrolling.

Clicking any thumbnail restores the system to the corresponding historical state: the recommendation panel repopulates with the candidates available at that step, the live preview reverts to that chart, and the parameter controls reflect the choices made up to that point. If the user then accepts a different recommendation from this restored state, the new choice is appended immediately to the right of the selected thumbnail, and all entries to its right that represent the previously explored path are discarded (DR5). This implements a *tree-structured* version history with a single active branch, making non-linear exploration tractable for novice users.

Code view. To help users track how each decision maps to the underlying specification (DR4), VisAutocomplete displays the Vega-Lite code corresponding to the current chart state (Figure 3f). The code view updates synchronously with the live preview: hovering over a recommendation card changes both the chart and the code simultaneously. Additions to the specification are highlighted in blue, and removals in red, making it immediately apparent how a candidate decision would modify the current state. Note that this view is optional rather than core to the novice workflow. It serves as a stepping stone for users who wish to learn Vega-Lite, and helps them locate where an error occurred when the tool produces an unexpected result.

5 Evaluation

We evaluate VisAutocomplete through a controlled user study. Below we describe the evaluation protocol.

5.1 Objectives

We evaluate the effectiveness of VisAutocomplete along two dimensions that reflect the core goals of our system (Section 3), by comparing it against widely used visualization authoring systems and automated chart recommendation tools. First, we verify that VisAutocomplete is at least as **approachable** as existing tools, given that its stepwise interface is designed to lower the barrier to entry for workers with low visualization literacy. Second, beyond ease of use, we ask whether

VisAutocomplete's stepwise approach actually improves users' **articulacy**, i.e., helps users produce visualizations that best delivers users' communicative intent. We establish three hypotheses:

H1 Charts produced with VisAutocomplete are of higher quality than those generated by baselines.

H2 The effect in H1 is amplified when authoring more complex charts (e.g., charts encoding a larger number of variables).

H3 The approachability of VisAutocomplete is comparable to that of baselines.

Here, we define charts as having higher quality if they more effectively convey their intended message. We also say a chart authoring tool is more approachable when users feel less cognitive load.

5.2 Study Design

To verify these hypotheses, we set two independent variables:

- **Visualization authoring tools**: *Visualization Autocomplete* and baseline techniques (*Excel*, *LLM vibecoding*, and *TaskVis* [42])
- **Complexity of charts**: *Simple* (visualizing two features in a dataset) and *Complex* (visualizing four features),

examining how these variables impact the quality of charts and approachability. The detailed study design is as follows:

Participants. Our target population is domain experts who are proficient with data but have little background in visualization design [58]—a group that constitutes a significant proportion of everyday chart creators [9]. We recruit 18 office workers (P1–P18) with experience creating charts for communicative purposes in their daily workflow, but without formal training in data visualization (10 males and eight females, aged 23–42 [30.5 ± 6.5]). The average experience of authoring visualizations ranges from 1 to 10 years ([4.3 ± 2.8]). We leverage the internal web community of three local universities and snowball sampling [10]. We constrain participants to have experience in creating charts for communicative purposes in their daily workflow. We compensate the participants with a gift card equivalent to 15 USD.

Assignment. For each trial, we provide participants with (1) a dataset, (2) a message, and (3) a visualization authoring tool. Then, we ask them to create a chart that effectively communicates the message to a general audience. We preprocess the dataset and remove features unrelated to the target message to minimize the influence of differences in data literacy on the study outcomes. We develop a website that allows participants to submit the final version of their visualization by uploading the relevant files or copy-pasting a screenshot.

Baseline techniques. We evaluate the superiority of VisAutocomplete relative to existing authoring tools that provide machine support. We select baselines that span a range of agency levels and approachability.

Microsoft Excel. We include Microsoft Excel as a baseline because it is the most widely used tool for data analysis and chart creation among office workers [6]. It is also a representative tool with relatively low user agency and high approachability. When using Excel, we provide data in both long and wide formats, and instruct basic functionalities like sort, filter, and transpose. We present participants with the "Insert" tab, which includes the "Recommended Charts" feature, and encourage them to use this functionality.

LLM vibecoding. We include LLM vibecoding (e.g., ChatGPT or Claude) as a baseline and instruct participants to use natural language prompts to generate visualizations. We add this baseline because prompt-based visualization authoring has become increasingly common [41, 61]. We want our baseline to best represent what office workers without visualization expertise would most commonly use. We select ChatGPT because it is currently one of the most widely used commercial LLM services [7, 35] [2] [3] . We also consider programming assistants like Codex or Claude Code, but exclude them because they are less commonly used by office workers and more likely to target expert programmers. We use the official service interface[4] with a ChatGPT Pro subscription. We use "developer mode" to disable memory functionality, constraining the LLM not to reference previous chat logs. We do not constrain the LLM's behavior, e.g., by using a system prompt, to simulate regular office work. We conduct the experiment from March 21st to 26th, 2026, where participants can freely select "Instant" or "Thinking" mode with GPT-5.3.

TaskVis. We include TaskVis [42] as a representative for an automated visualization recommendation engine. We selected TaskVis over alternative systems such as Draco [26] and Dziban [20]. This is because it allows the analytical tasks that a visualization should support (e.g., "correlate" or "compare") to be specified explicitly as an input parameter, which aligns well with our study design, where participants create charts based on a given message. Because TaskVis is a fully automated system and also requires users to understand its syntax, we do not ask participants to use it directly. Instead, we execute it ourselves by translating the prepared chart messages into corresponding tasks and passing them to the TaskVis module as input parameters.

Stimuli design. We design our stimuli (chart messages and corresponding datasets) to simulate everyday analytic contexts, or situations in which an office worker creates charts to communicate messages to the general public or to coworkers with limited data literacy. To this end, we derive stimuli by examining visualizations published in news articles. The detailed procedure is as follows:

(Step 1) Investigating reference dataset. We select news visualizations from the MASSVIS dataset [3] as our reference corpus. To minimize bias and reduce manual effort, we extract 200 semantically diverse charts via stratified sampling [1, 30] (see Appendix A for details) from the corpus. We then manually analyze these charts, extracting (1) the domain, (2) the number of variables, (3) the intended message, and (4) the analytical tasks required to decode the message. Two coders independently analyze the charts and reconcile their annotations through two discussion sessions. Initial inter-coder agreement, measured by Cohen's κ, is 0.58. Note that while classifying analytical tasks, we refer to Quadri and Rosen's taxonomy on visualization tasks [32].

(Step 2) Selecting analytical tasks. To determine the analytical tasks that participants' charts should support, we analyzed the frequency of tasks in our reference dataset. As a result, we select `characterize distribution`, `sort`, and `correlate` as representative analytical tasks, as they are (1) the most commonly appearing tasks in the corpus, and (2) frequently used in isolation without being paired with other tasks. Although `compare` is the most frequent task in our corpus, it is inherently combinatorial; it rarely appears in isolation but instead augments other tasks (e.g., comparing distributions, comparing correlations). We therefore exclude it from simple charts, where each trial involves a single analytical task, and incorporate it into complex charts, where multiple tasks are combined. Hence, each participant completed all combinations of analytical tasks and independent variables, resulting in 3 [analytical tasks] $\times$ 3 [authoring tools] $\times$ 2 [complexity levels] = 18 trials per participant.

(Step 3) Selecting datasets and chart messages. The intent behind selecting datasets and messages is to ensure semantic diversity across chart messages, to minimize the influence of participants' domain background knowledge on the experimental results. We adopt nine major news topics from The New York Times[5] (Domestic, World, Business, Lifestyle, Art, Sports, Games, Cooking, and Economy). For each analytical task (`characterize distribution`, `sort`, and `correlate`), we randomly assign three non-overlapping topics.

For each topic-task pair, we then collect datasets suitable for creating visualizations that support the assigned analytical task. To do so, we first manually collect 30 datasets per topic from public repositories such as Kaggle. We then filter out datasets that are not suitable for the assigned task (Appendix A for detailed criteria). For example, for `correlate`, we retain only datasets containing at least two quantitative variables with a Pearson correlation greater than 0.5. We then examine

[2] https://www.techbusinessnews.com.au/...
[3] https://www.reuters.com/...
[4] https://chatgpt.com/
[5] https://www.nytimes.com/international/

the remaining datasets and curate candidate chart messages that can be derived from each dataset. From these, we select one message and its corresponding dataset for a simple chart and another for a complex chart. When multiple candidate messages are available, we prioritize messages that can be expressed through a diverse range of chart designs. Messages for simple and complex charts involve two and four variables, respectively. These correspond to the 25th and 75th percentiles of the variable count distribution in our reference corpus. The former reflects commonly created charts and the latter represents more demanding charts that office workers may occasionally encounter.

Procedure. After participants sign the consent form, we explain the study purpose and tasks. Participants then complete three sessions; each session comprises six trials in which they author charts using one of the three tools (VisAutocomplete, Excel, and an LLM). We counterbalance the tool order across participants using a full-factorial design. After each session, participants are guided to evaluate the approachability of the tools using NASA-TLX and the system usability scale (SUS). Within each session, three trials require participants to create complex charts and the other three require simple charts. We fully randomize trial order to counterbalance complexity. We counterbalance the assignment of chart messages across sessions using a Latin-square design. We conduct a post-hoc interview after all sessions are finished. All studies have finished within 90 minutes.

Evaluating articulacy. We evaluate chart quality as a proxy for articulacy. To this end, we recruit eight visualization experts to assess the charts created with each tool (all male, aged 27–45, 35.6 ± 6.4). All evaluators hold a Ph.D. degree, including three professors, four postdoctoral researchers, and one industry researcher. Their primary research area is data visualization, where they regularly publish in major visualization venues such as TVCG and VIS. On average, they have 10.1 years of experience in visualization research (± 3.8).

After they sign the consent form, we provide evaluators with the charts made by study participants and the corresponding message. Aligned with our definition of articulacy, we ask them to determine the articulacy based on the degree to which the chart effectively and accurately conveys the message to general audiences. We show each pair of charts and messages individually, and guide evaluators to answer the Likert scale questionnaire spanning from 1 (very poor) to 7 (very effective). All evaluations have finished within 40 minutes. We compensate each participant with a USD 20 gift card.

We sampled 40 charts per tool (160 total) using stratified sampling to balance topic and complexity, with each chart rated by three experts, yielding 480 evaluations (120 per tool).

6 Results

We present our quantitative evaluation results on articulacy and approachability. We then discuss the usage patterns of VisAutocomplete, as well as the qualitative findings from the post-hoc interview and charts created by study participants.

6.1 Quantitative Results

We discuss our quantitative findings on articulacy and approachability in using VisAutocomplete and baselines.

Articulacy. We conduct a two-way Aligned Rank Transform (ART) ANOVA to examine the effects of the independent variables, visualization authoring tool and chart complexity, on chart quality. We use ART ANOVA because chart quality is assessed with Likert-scale questionnaires, making a nonparametric analysis appropriate. We also use ART contrast tests with Holm correction for post-hoc analysis.

As a result (Figure 4), VisAutocomplete consistently produces higher-quality charts than TaskVis and Excel, and further outperforms LLM vibecoding when authoring complex charts (confirms H1, H2).

We find that the selection of visualization authoring tools significantly alters the chart quality ($F_{3,143.73} = 77.62$, $p < .001$). Post hoc analysis shows that both VisAutocomplete and the LLM vibecoding produced significantly higher-quality charts than Excel and TaskVis ($p < .001$ for all comparisons). We also find that chart complexity significantly affects chart quality ($F_{1,145.93} = 42.03$, $p < .001$), with

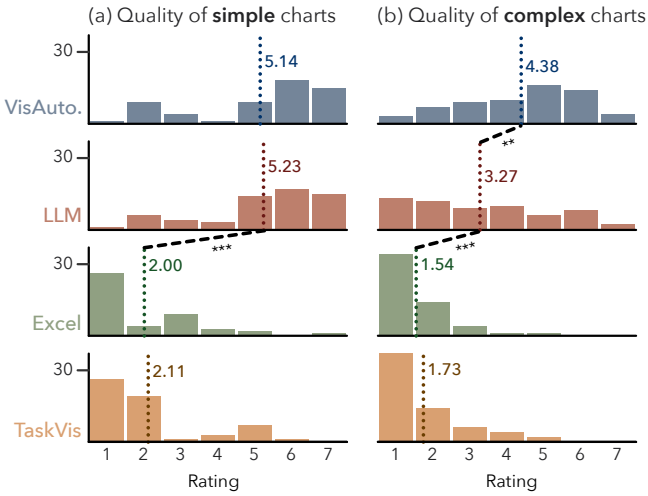

Fig. 4: **Articulacy of charts created by participants.** VisAutocomplete produced higher-articulacy charts than Excel and TaskVis, and outperformed LLM vibecoding for complex charts (***: $p < .001$, **: $p < .01$).

complex charts receiving lower scores than simple ones ($p < .001$). Furthermore, the interaction between authoring tool and chart complexity is also statistically significant ($F_{3,144.16} = 5.59$, $p < .01$). To further examine this interaction, we conduct separate one-way ART ANOVAs on visualization authoring tool for simple and complex charts. For simple charts, the ANOVA and post hoc results are consistent with those for the full dataset. For complex charts, however, VisAutocomplete outperforms not only Excel and TaskVis ($p < .001$ for both), but also the LLM vibecoding ($p < .01$) in chart quality.

Approachability as usability scores. We conduct one-way ART ANOVAs to examine (1) SUS scores, (2) average NASA-TLX scores, and (3) scores on individual NASA-TLX items. For consistency in interpretation, we reverse the `performance` item, as lower values originally indicate better performance.

As a result (Figure 6), VisAutocomplete and the LLM vibecoding achieve comparable usability, both outperforming Excel, though the LLM imposes greater temporal demand than VisAutocomplete (confirms H3). The selection of authoring tools significantly changes both SUS scores ($F_{2,34} = 15.93$, $p < .001$) and average NASA-TLX scores ($F_{2,34} = 19.30$, $p < .001$). Post hoc analysis shows a similar pattern for both measures: VisAutocomplete and the LLM vibecoding exhibit comparable usability, and both outperform Excel ($p < .001$ for all pairwise comparisons). This overall pattern is also largely consistent across the individual NASA-TLX items. However, the factors underlying reduced usability differ between VisAutocomplete and the LLM vibecoding. In particular, the LLM vibecoding imposes significantly higher temporal demand compared to VisAutocomplete ($p < .01$).

Task completion time. We conduct a one-way repeated measure (RM) ANOVA to examine differences in task completion time across visualization authoring tools. As a result, we find no significant difference in completion time by authoring tools ($F_{2,321} = 0.368$, $p = 0.692$).

6.2 Usage Patterns

We provide two analysis pertaining to user agency in VisAutocomplete.

Reliance on recommendation rank. To understand how much users rely on the order of recommended charts, we examined which rank they most often selected (see Section 4.4) across three recommendation categories, including encoding, data transformation, and style. The results are shown in Figure 7. Participants did not simply accept the recommendations presented to them: they declined the top-ranked recommendation in roughly three out of every seven selections. Instead, they exercised different degrees of agency depending on the authoring context. Because recommendations for data transformation and style often involve more fine-grained design decisions, participants demonstrated greater agency when refining such details, becoming less likely

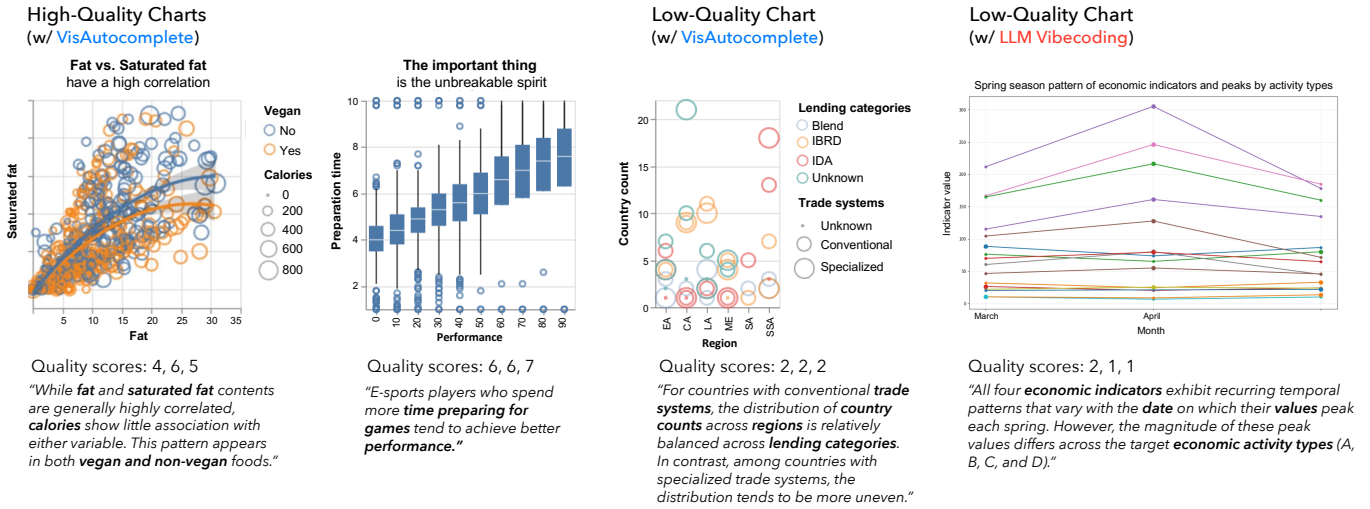

Fig. 5: **Example charts designed by participants in our user study.** The quality scores of each chart evaluated by visualization experts are shown below the chart, followed by its intended message in italics. Note that the study is conducted in Korean, and the chart text shown in the figure has been translated into English from the original outputs.

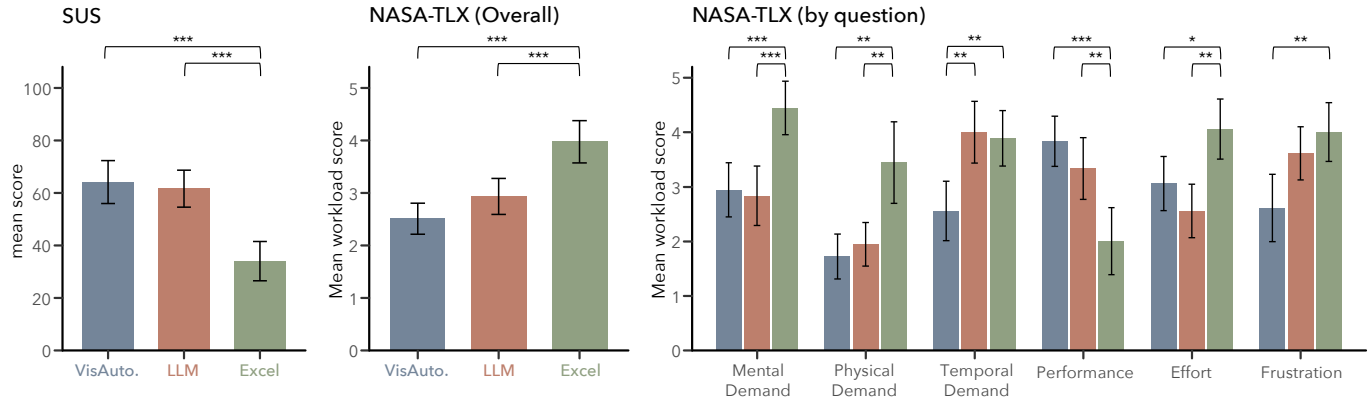

Fig. 6: **Usability of visualization authoring tools measured using SUS and NASA-TLX.** Although VisAutocomplete and the LLM vibecoding exhibited similar overall usability, the factors that reduced their usability differ (***: $p < .001$, **: $p < .01$, *: $p < .05$).

to follow high-ranked recommendations in these categories.

We believe this shift in the dominance of the first recommendation follows from coverage in our dataset. Because encoding primitives appear in nearly every chart (e.g., x- and y-axes), our co-occurrence statistics for encoding are well-estimated, so the top-ranked recommendation more reliably matches what users want. Style and data-transformation primitives, in contrast, vary more across charts and thus occur less often individually. With less data to estimate their co-occurrence patterns, the top-ranked recommendation is more likely to diverge from an individual's actual preference, making participants less likely to follow it.

Restoration behavior. To further examine how actively participants engaged with the tool, we counted the number of restorations, i.e., instances where participants clicked a thumbnail in the graphical history view to revert to a previous state (see Section 4.4).

Figure 8 shows the average number of iterations per chart for each participant. Restoration use varied substantially across participants: six never restored a previous version, while those who did so restored 1.79 times on average. This echoes our analysis of card ranking (above): some participants followed the system's suggested trajectory, while others actively revised it, differing in their desired level of agency. VisAutocomplete accommodated both, allowing users to intervene to the extent they wanted.

6.3 Post-study Interview Results

We discuss notable qualitative results from the post-hoc interview.

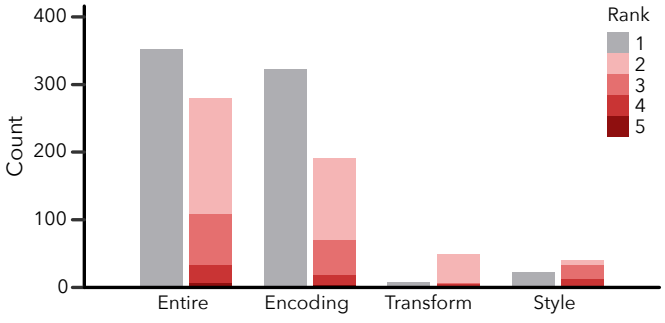

Fig. 7: **Distribution of the ranks of recommendation cards selected by participants in our user study.** The degree to which participants followed the top-ranked (rank-1) recommendation decreased for data transformation and style.

Importance of responsiveness. Ten of the 18 participants (56%) note that the long response time of the LLM made them hesitant to iteratively refine the output visualization. Among them, three explicitly identified latency as the primary source of frustration when the LLM produced a low-quality visualization. In contrast, four participants (22%) report that VisAutocomplete's responsiveness motivated them to explore a wider range of design alternatives. For example, P3 remark, *"I liked that the visualization changed immediately whenever I hovered over something... That pushes me to explore a variety of options."*

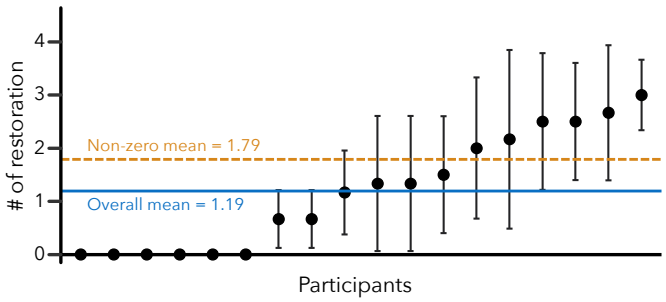

Fig. 8: **The average number of restoration made in our experiments by participants.** The degree which participants restore the previous version of charts vary substantially by individual.

Needs for explainability. Participants identified two limitations related to explainability. First, four out of the 18 participants (22%) report that they found it difficult to accept VisAutocomplete's recommendations because the system did not explain why those recommendations are made. Second, six participants (33%) note that the system provided insufficient explanation of key terms such as mark, encoding, and transform, suggesting that the interface may be difficult to use for users with low visualization literacy.

Increased efforts with VisAutocomplete. 14 of the 18 participants (77%) report that they invest the greatest effort when using VisAutocomplete among the three visualization authoring systems, whereas the remaining four pick the LLM vibecoding. Among those participants, five mention that the visual presentation of diverse design options induced them to do so. For example, P11 remarks, *"As I explored various design options using hovering, I suddenly found myself thinking more deeply about which option leads to a good chart design."* This finding is consistent with our quantitative results that charts created with VisAutocomplete achieve higher quality (Section 6.1).

7 Discussion

Below we discuss issues on stepwise visualization autocompletion.

7.1 Approachability and Articulacy

On approachability, VisAutocomplete performed comparably to LLM vibecoding and significantly better than Excel, whose limited recommendation scope and lack of previews left users with little support for exploration, causing them to disengage earlier. Although VisAutocomplete and the LLM showed no significant difference on SUS or NASA-TLX, interviews revealed qualitatively different experiences. VisAutocomplete required more effort to use, while LLM vibecoding brought greater time pressure, with participants noting that *"waiting for ChatGPT to generate a chart takes too long."*

On articulacy, VisAutocomplete and the LLM performed similarly for simple charts, where the design space is small enough that a single well-generated attempt often lands near an acceptable solution. For complex charts, however, VisAutocomplete significantly outperformed the LLM and produced fewer lowest-scoring charts, an effect we attribute to the stepwise interface supporting broader exploration at low latency, rather than committing users to a single generated result (see Figure 5 for failure cases). TaskVis underperformed relative to all interactive tools, likely due to its inability to capture task intent without user input (see Appendix B).

7.2 Chart Authoring as Iterative Search

Our results and interviews suggest that users approach chart authoring as an iterative search rather than a one-shot specification task. Several participants could recognize when Excel's or the LLM's output did not match their intent, even when they could not articulate why or produce a better alternative themselves. This suggests that for novices, the challenge may be less about judgment than production. Evaluating designs may come more naturally than generating them.

Because users may not be able to directly express their ideal chart, finding it likely requires *exploration*. VisAutocomplete supports this by decomposing the design process into sequential steps grounded in common practice, turning authoring into a series of recognition judgments rather than an open-ended generative problem. Consistent with this, participants declined top-ranked recommendations in roughly three of seven selections and varied widely in restoration behavior (Section 6.2), and this exploratory behavior translated directly into higher-quality outcomes for complex charts. Giving designers more options to consider, without imposing additional burden, thus helps them find charts that better match their communicative intent.

7.3 The Impact of Latency in Iterative Design

A consistent theme across our results is that responsiveness shapes not just user experience, but the iterative design process itself. When each iteration is costly, users settle early, accepting a chart that does not fully match their intent rather than exploring further. LLM latency discouraged re-iteration after a poor first attempt; as P3 notes, *"I felt deincentivized to iterate with GPT... So I tried to put every possible effort into an initial attempt."*

A slow tool does not just frustrate users; it curtails the iterations they are willing to make, and therefore how thoroughly they explore the design space. VisAutocomplete's responsive stepwise interface addressed this by making each step fast and lightweight, lowering the cost of exploration until browsing felt natural rather than burdensome. In iterative design, latency appears to impact design quality, not just user experience.

7.4 Stepwise Design Raises the Articulacy Lower Bound

Beyond producing charts with higher articulacy on average, VisAutocomplete produced notably fewer low-articulacy charts in the complex condition compared to LLM vibecoding. We attribute this to a structural property of the stepwise process: comparing several options before committing, rather than receiving a single generated result.

LLM vibecoding, by contrast, delivers a complete chart in a single step. If that chart is poor and latency discourages re-iteration, the articulacy of the final chart is bounded by that first attempt. As one participant (P11) notes, *"If ChatGPT gets the chart wrong the first time, I don't want to try again. It just takes too long."* Stepwise design raises this lower bound by presenting several recommendations at each step for users to compare. Notably, this persisted even with our suboptimal, coverage-limited scoring (Section 6.2), suggesting the interaction itself, not scoring precision, is what matters.

7.5 Limitations and Future Work

We emphasize that our stepwise design recommendation workflow opens up interesting future work directions. The first direction concerns a new form of design feedback that our decomposition makes possible. Because the process is decomposed into primitive steps, feedback can target each decision as it is made, rather than only evaluating completed charts as most existing tools do [46]. This matters because preview-based selection alone proved insufficient in our study. Participants browsed options without developing the confidence to commit. Step-level feedback is one way to build that confidence, though designing it well raises open challenges. It must strengthen decision-making rather than just report on it. It must also keep cognitive load and interruption low. And it should ultimately advance visualization literacy so that users grow less dependent on it over time.

The second direction concerns improving the components in VisAutocomplete. While VisAutocomplete outperforms existing tools for complex charts, articulacy still declined for complex charts relative to simple ones. This was particularly true when the large number of variables and encoding choices led novices to suboptimal decisions (see Figure 5). This is consistent with Section 6.2, where reliance on the top-ranked recommendation dropped precisely for the sparsely covered style and transformation categories. Addressing these will likely require richer models of design intent that move beyond co-occurrence frequency toward representations capturing a visualization's communicative goals. Pursuing these directions would bring us closer to enabling users without visualization expertise to author effective charts.

Acknowledgments

This work was supported partly by Villum Investigator grant VL-54492 by Villum Fonden. Any opinions, findings, and conclusions expressed in this material are those of the authors and do not necessarily reflect the views of the funding agency.

References

[1] M. M. Abbas, M. Aupetit, M. Sedlmair, and H. Bensmail. ClustMe: A visual quality measure for ranking monochrome scatterplots based on cluster patterns. *Computer Graphics Forum*, 38(3):225–236, 2019. doi: 10.1111/cgf.13684 6

[2] S. Amershi, D. S. Weld, M. Vorvoreanu, A. Fourney, B. Nushi, P. Collisson, J. Suh, S. T. Iqbal, P. N. Bennett, K. Inkpen, J. Teevan, R. Kikin-Gil, and E. Horvitz. Guidelines for Human-AI interaction. In *Proceedings of the ACM Conference on Human Factors in Computing Systems*, pp. 3:1–3:13. ACM, New York, NY, USA, 2019. doi: 10.1145/3290605.3300233 2

[3] M. A. Borkin, A. A. Vo, Z. Bylinskii, P. Isola, S. Sunkavalli, A. Oliva, and H. Pfister. What makes a visualization memorable? *IEEE Transactions on Visualization and Computer Graphics*, 19(12):2306–2315, 2013. doi: 10.1109/TVCG.2013.234 6

[4] M. Brehmer and T. Munzner. A multi-level typology of abstract visualization tasks. *IEEE Transactions on Visualization and Computer Graphics*, 19(12):2376–2385, 2013. doi: 10.1109/TVCG.2013.124 2

[5] S. K. Card, J. Mackinlay, and B. Shneiderman. *Readings in Information Visualization: Using Vision to Think*. Morgan Kaufmann, San Francisco, CA, USA, 1999. 5

[6] G. Chalhoub and A. Sarkar. "It's freedom to put things where my mind wants": Understanding and improving the user experience of structuring data in spreadsheets. In *Proceedings of the ACM Conference on Human Factors in Computing Systems*, pp. 585:1–585:24. ACM, New York, NY, USA, 2022. doi: 10.1145/3491102.3501833 6

[7] Y. Chen, S. N. Kirshner, A. Ovchinnikov, M. Andiappan, and T. Jenkin. A manager and an AI walk into a bar: Does ChatGPT make biased decisions like we do? *Manufacturing & Service Operations Management*, 27(2):354–368, 2025. doi: 10.1287/msom.2023.0279 6

[8] J. J. Darragh, I. H. Witten, and M. L. James. The reactive keyboard: A predicive typing aid. *Computer*, 23(11):41–49, 1990. doi: 10.1109/2.60879 3

[9] Data Visualization Society. Data visualization society state of the industry report, 2024. [Online]. Available: https://www.datavisualizationsociety.org/soti-report-2024. 2, 6

[10] L. A. Goodman. Snowball sampling. *The Annals of Mathematical Statistics*, 32(1):148–170, 1961. 6

[11] L. Grammel, M. Tory, and M.-A. Storey. How information visualization novices construct visualizations. *IEEE Transactions on Visualization and Computer Graphics*, 16(6):943–952, 2010. doi: 10.1109/TVCG.2010.164 1, 2

[12] J. Heer. Agency plus automation: Designing artificial intelligence into interactive systems. *Proceedings of the National Academy of Sciences*, 116(6):1844–1850, 2019. doi: 10.1073/PNAS.1807184115 2

[13] J. Heer, J. D. Mackinlay, C. Stolte, and M. Agrawala. Graphical histories for visualization: Supporting analysis, communication, and evaluation. *IEEE Transactions on Visualization and Computer Graphics*, 14(6):1189–1196, 2008. doi: 10.1109/TVCG.2008.137 5

[14] A. K. Hopkins, M. Correll, and A. Satyanarayan. Visualint: Sketchy in situ annotations of chart construction errors. *Computer Graphics Forum*, 39(3):219–228, 2020. doi: 10.1111/cgf.13975 2

[15] E. Horvitz. Principles of mixed-initiative user interfaces. In *Proceedings of the ACM Conference on Human Factors in Computing Systems*, pp. 159–166. ACM, New York, NY, USA, 1999. doi: 10.1145/302979.303030 2

[16] X. Hou, Y. Zhao, S. Wang, and H. Wang. Model context protocol (mcp): Landscape, security threats, and future research directions. *ACM Transactions on Software Engineering Methodology*, Feb. 2026. doi: 10.1145/3796519 4

[17] K. Z. Hu, M. A. Bakker, S. Li, T. Kraska, and C. A. Hidalgo. VizML: A machine learning approach to visualization recommendation. In *Proceedings of the ACM Conference on Human Factors in Computing Systems*, pp. 128:1–128:12. ACM, New York, NY, USA, 2019. doi: 10.1145/3290605.3300358 2

[18] Y. Kim, K. Wongsuphasawat, J. Hullman, and J. Heer. GraphScape: A model for automated reasoning about visualization similarity and sequencing. In *Proceedings of the ACM Conference on Human Factors in Computing Systems*, pp. 2628–2638. ACM, New York, NY, USA, 2017. doi: 10.1145/3025453.3025866 3

[19] H.-K. Ko, H. Jeon, G. Park, D. H. Kim, N. W. Kim, J. Kim, and J. Seo. Natural language dataset generation framework for visualizations powered by large language models. In *Proceedings of the ACM Conference on Human Factors in Computing Systems*, pp. 843:1–843:22. ACM, New York, NY, USA, 2024. doi: 10.1145/3613904.3642943 3

[20] H. Lin, D. Moritz, and J. Heer. Dziban: Balancing agency & automation in visualization design via anchored recommendations. In *Proceedings of the ACM Conference on Human Factors in Computing Systems*, pp. 751:1–751:12. ACM, New York, NY, USA, 2020. doi: 10.1145/3313831.3376880 1, 2, 6

[21] Z. Liu, J. Thompson, A. Wilson, M. Dontcheva, J. Delorey, S. Grigg, B. Kerr, and J. T. Stasko. Data illustrator: Augmenting vector design tools with lazy data binding for expressive visualization authoring. In *Proceedings of the ACM Conference on Human Factors in Computing Systems*, pp. 123:1–123:13. ACM, New York, NY, USA, 2018. doi: 10.1145/3173574.3173697 2

[22] S. L'Yi, A. van den Brandt, E. Adams, H. N. Nguyen, and N. Gehlenborg. Learnable and expressive visualization authoring through blended interfaces. *IEEE Transactions on Visualization and Computer Graphics*, 31(1):459–469, 2025. doi: 10.1109/TVCG.2024.3456598 2

[23] P. Maes. Intelligent software: easing the burdens that computers put on people. *IEEE Expert*, 11(6):62–63, Dec 1996. doi: 10.1109/64.546584 2

[24] A. McNutt, G. Kindlmann, and M. Correll. Surfacing visualization mirages. In *Proceedings of the ACM Conference on Human Factors in Computing Systems*, pp. 293:1–293:16. ACM, New York, NY, USA, 2020. doi: 10.1145/3313831.3376420 2

[25] G. G. Méndez, U. Hinrichs, and M. A. Nacenta. Bottom-up vs. top-down: Trade-offs in efficiency, understanding, freedom and creativity with infovis tools. In *Proceedings of the ACM Conference on Human Factors in Computing Systems*, pp. 841–852. ACM, New York, NY, USA, 2017. doi: 10.1145/3025453.3025942 1

[26] D. Moritz, C. Wang, G. L. Nelson, H. Lin, A. M. Smith, B. Howe, and J. Heer. Formalizing visualization design knowledge as constraints: Actionable and extensible models in Draco. *IEEE Transactions on Visualization and Computer Graphics*, 25(1):438–448, 2019. doi: 10.1109/TVCG.2018.2865240 1, 2, 6

[27] T. Munzner. A nested model for visualization design and validation. *IEEE Transactions on Visualization and Computer Graphics*, 15(6):921–928, Nov 2009. doi: 10.1109/TVCG.2009.111 2

[28] T. Munzner. *Visualization Analysis and Design*. A.K. Peters Visualization Series. CRC Press, Boca Raton, FL, USA, 2014. 2

[29] A. Narechania, A. Srinivasan, and J. Stasko. NL4DV: A toolkit for generating analytic specifications for data visualization from natural language queries. *IEEE Transactions on Visualization and Computer Graphics*, 27(2):369–379, 2021. doi: 10.1109/TVCG.2020.3030378 3

[30] A. V. Pandey, J. Krause, C. Felix, J. Boy, and E. Bertini. Towards understanding human similarity perception in the analysis of large sets of scatter plots. In *Proceedings of the ACM Conference on Human Factors in Computing Systems*, p. 3659–3669. ACM, New York, NY, USA, 2016. doi: 10.1145/2858036.2858155 6

[31] P. Parsons. Understanding data visualization design practice. *IEEE Transactions on Visualization and Computer Graphics*, 28(1):665–675, 2022. doi: 10.1109/TVCG.2021.3114959 2

[32] G. J. Quadri and P. Rosen. A survey of perception-based visualization studies by task. *IEEE Transactions on Visualization and Computer Graphics*, 28(12):5026–5048, 2022. doi: 10.1109/TVCG.2021.3098240 6

[33] D. Ren, T. Höllerer, and X. Yuan. iVisDesigner: Expressive interactive design of information visualizations. *IEEE Transactions on Visualization and Computer Graphics*, 20(12):2092–2101, 2014. doi: 10.1109/TVCG.2014.2346291 2

[34] D. Ren, B. Lee, and M. Brehmer. Charticulator: Interactive construction of bespoke chart layouts. *IEEE Transactions on Visualization and Computer Graphics*, 25(1):789–799, 2019. doi: 10.1109/TVCG.2018.2865158 2

[35] J. Russell, M. Karpinska, and M. Iyyer. People who frequently use ChatGPT for writing tasks are accurate and robust detectors of AI-generated text. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics*, pp. 5342–5373. Association for Computational Linguistics, Vienna, Austria, July 2025. doi: 10.18653/v1/2025.acl-long.267 6

[36] A. Satyanarayan and J. Heer. Lyra: An interactive visualization design

environment. *Computer Graphics Forum*, 33(3):351–360, 2014. doi: 10.1111/CGF.12391 1, 2

[37] A. Satyanarayan, B. Lee, D. Ren, J. Heer, J. T. Stasko, J. Thompson, M. Brehmer, and Z. Liu. Critical reflections on visualization authoring systems. *IEEE Transactions on Visualization and Computer Graphics*, 26(1):461–471, 2020. doi: 10.1109/TVCG.2019.2934281 2

[38] A. Satyanarayan, D. Moritz, K. Wongsuphasawat, and J. Heer. Vega-lite: A grammar of interactive graphics. *IEEE Transactions on Visualization and Computer Graphics*, 23(1):341–350, 2017. doi: 10.1109/TVCG.2016.2599030 3

[39] D. A. Schön. *The Reflective Practitioner: How Professionals Think in Action*. Routledge, 2017. 2

[40] M. Sedlmair, M. Meyer, and T. Munzner. Design study methodology: Reflections from the trenches and the stacks. *IEEE Transactions on Visualization and Computer Graphics*, 18(12):2431–2440, 2012. doi: 10.1109/TVCG.2012.213 2

[41] L. Shen, E. Shen, Y. Luo, X. Yang, X. Hu, X. Zhang, Z. Tai, and J. Wang. Towards natural language interfaces for data visualization: A survey. *IEEE Transactions on Visualization and Computer Graphics*, 29(6):3121–3144, 2023. doi: 10.1109/TVCG.2022.3148007 6

[42] L. Shen, E. Shen, Z. Tai, Y. Song, and J. Wang. TaskVis: Task-oriented visualization recommendation. In *Short Paper Proceedings of the Eurographics/IEEE VGTC Conference on Visualization*, pp. 91–95. Eurographics Association, 2021. doi: 10.2312/EVS.20211061 1, 2, 6

[43] T. B. Sheridan. *Telerobotics, Automation, and Human Supervisory Control*. MIT Press, Cambridge, MA, USA, 1992. 2

[44] S. Shin, S. Chung, S. Hong, and N. Elmqvist. A scanner deeply: Predicting gaze heatmaps on visualizations using crowdsourced eye movement data. *IEEE Transactions on Visualization and Computer Graphics*, 29(1):396–406, Jan 2023. doi: 10.1109/TVCG.2022.3209472 2

[45] S. Shin, S. Hong, and N. Elmqvist. Perceptual Pat: A virtual human visual system for iterative visualization design. In *Proceedings of the ACM Conference on Human Factors in Computing Systems*, pp. 811:1–811:17. ACM, New York, NY, USA, 2023. doi: 10.1145/3544548.3580974 2

[46] S. Shin, S. Hong, and N. Elmqvist. Visualizationary: Automating design feedback for visualization designers using large language models. *IEEE Transactions on Visualization and Computer Graphics*, 31(10):8796–8813, Oct 2025. doi: 10.1109/TVCG.2025.3579700 2, 9

[47] B. Shneiderman. Direct manipulation: A step beyond programming languages. *Computer*, 16(8):57–69, 1983. doi: 10.1109/MC.1983.1654471 3

[48] B. Shneiderman. *Human-Centered AI*. Oxford University Press, Oxford, United Kingdom, 2022. 2

[49] Y. Tian, W. Cui, D. Deng, X. Yi, Y. Yang, H. Zhang, and Y. Wu. Chartgpt: Leveraging llms to generate charts from abstract natural language. *IEEE Transactions on Visualization and Computer Graphics*, 31(3):1731–1745, 2025. doi: 10.1109/TVCG.2024.3368621 1, 3

[50] S. van den Elzen and J. J. van Wijk. Small multiples, large singles: A new approach for visual data exploration. *Comput. Graph. Forum*, 32(3):191–200, 2013. doi: 10.1111/CGF.12106 2

[51] J. VanderPlas, B. E. Granger, J. Heer, D. Moritz, K. Wongsuphasawat, A. Satyanarayan, E. Lees, I. Timofeev, B. Welsh, and S. Sievert. Altair: Interactive statistical visualizations for Python. *Journal of Open Source Software*, 3(32):1057, 2018. doi: 10.21105/joss.01057 3

[52] C. Wang, Y. Feng, R. Bodík, I. Dillig, A. Cheung, and A. J. Ko. Falx: Synthesis-powered visualization authoring. In *Proceedings of the ACM Conference on Human Factors in Computing Systems*, pp. 106:1–106:15. ACM, New York, NY, USA, 2021. doi: 10.1145/3411764.3445249 1

[53] C. Wang, J. Thompson, and B. Lee. Data formulator: Ai-powered concept-driven visualization authoring. *IEEE Transactions on Visualization and Computer Graphics*, 30(1):1128–1138, 2024. doi: 10.1109/TVCG.2023.3326585 2

[54] L. Wang, S. Zhang, Y. Wang, E.-P. Lim, and Y. Wang. LLM4Vis: Explainable visualization recommendation using ChatGPT. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing: Industry Track*, pp. 675–692. Association for Computational Linguistics, Singapore, Dec. 2023. doi: 10.18653/v1/2023.emnlp-industry.64 1

[55] J. Wei, X. Wang, D. Schuurmans, M. Bosma, b. ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, vol. 35, pp. 24824–24837. Curran Associates, Inc., 2022. 4

[56] W. Wei, S. Huron, and Y. Jansen. Towards autocomplete strategies for visualization construction. In *Short Paper Proceedings of the IEEE VIS Conference*, pp. 141–145. IEEE, 2023. doi: 10.1109/VIS54172.2023.00037 3

[57] L. Wilkinson. The grammar of graphics. In *Handbook of computational statistics: Concepts and methods*, pp. 375–414. Springer, 2011. 4

[58] Y. L. Wong, K. Madhavan, and N. Elmqvist. Towards characterizing domain experts as a user group. In *Proceedings of the IEEE VIS Workshop on Evaluation and Beyond - Methodological Approaches for Visualization*, pp. 1–10. IEEE Computer Society, Los Alamitos, CA, USA, 2018. doi: 10.1109/BELIV.2018.8634026 2, 6

[59] K. Wongsuphasawat, D. Moritz, A. Anand, J. Mackinlay, B. Howe, and J. Heer. Voyager: Exploratory analysis via faceted browsing of visualization recommendations. *IEEE Transactions on Visualization and Computer Graphics*, 22(1):649–658, 2016. doi: 10.1109/TVCG.2015.2467191 2, 3

[60] K. Wongsuphasawat, Z. Qu, D. Moritz, R. Chang, F. Ouk, A. Anand, J. Mackinlay, B. Howe, and J. Heer. Voyager 2: Augmenting visual analysis with partial view specifications. In *Proceedings of the ACM Conference on Human Factors in Computing Systems*, p. 2648–2659. ACM, New York, NY, USA, 2017. doi: 10.1145/3025453.3025768 1, 2

[61] Y. Wu, Y. Wan, H. Zhang, Y. Sui, W. Wei, W. Zhao, G. Xu, and H. Jin. Automated data visualization from natural language via large language models: An exploratory study. *Proc. ACM Manag. Data*, 2(3), article no. 115, 28 pages, May 2024. doi: 10.1145/3654992 6

[62] J. Zong, D. Barnwal, R. Neogy, and A. Satyanarayan. Lyra 2: Designing interactive visualizations by demonstration. *IEEE Transactions on Visualization and Computer Graphics*, 27(2):304–314, 2021. doi: 10.1109/TVCG.2020.3030367 1, 2

[63] J. Zong, J. Pollock, D. Wootton, and A. Satyanarayan. Animated VegaLite: Unifying animation with a grammar of interactive graphics. *IEEE Transactions on Visualization and Computer Graphics*, 29(1):149–159, 2023. doi: 10.1109/TVCG.2022.3209369 3